Q1. Core Functional Modules in the Business Logic Layer and Their UI Interactions

The Business Logic Layer (BLL) of the Smart Traffic Signal system acts as the core decision-making brain of the application. It processes inputs from the data layer (video feeds, AI models) and applies traffic management rules before sending the processed state to the Presentation Layer (UI).

1. Core BLL Modules

- **Traffic Signal Control Logic:** Controls the RED-YELLOW-GREEN state transitions, enforces sequential clockwise ordering , and applies adaptive timing based on vehicle density.
- **Emergency Handling Logic:** Detects the approach of emergency vehicles (e.g., ambulances), maintains a First-Come-First-Serve (FCFS) emergency queue, and requests signal preemption to safely interrupt the normal cycle.
- **Intersection & Vehicle-Flow Decision Logic:** Computes the number of queued vehicles per side, applies stop-line rules, enforces minimum following distances, and dictates per-vehicle movement frame-by-frame.
- **Vehicle Detection & Classification Logic:** Runs YOLO AI inference and translates raw model outputs into domain-specific data (total vehicle counts, categorizing cars, trucks, motorcycles, and emergency vehicles).
- **Logging & Violation Logic:** Records signal changes, traffic counts, and emergency events, while also logging lane violations into a MySQL database.

2. Interaction with the Presentation Layer (UI)

The BLL interacts seamlessly with the previously implemented presentation components. The interaction flow is structured as follows:

- The main application loop reads data inputs and feeds them into the BLL.
- The traffic_controller.update() module (BLL) calculates the active side, countdown timers, and switch reasons.
- The BLL pushes these processed states to the GUI update methods (e.g., update_signal_state(), update_timer(), update_vehicle_info()).
- The UI layers then render these outputs visually:
    - **Video Overlays:** Vehicle detector outputs are drawn as bounding boxes on the video panels.
    - **Signal Status:** Controller states are rendered as physical signal lights, countdown texts, and queue statistics in the Pygame interface.
    - **Alerts:** Emergency handler states trigger visual emergency banners and indicators on the dashboard.

Q2. Implementation of Business Rules, Validation, and Transformation

A) Implementation of Business Rules

Business rules dictate the specific conditions the traffic system must follow. They are enforced across different modules:

- **Signal Sequencing Rule:** Enforces that only one side receives a GREEN light at a time, strictly following a clockwise rotation.
- **Timing Constraints Rule:** Ensures that green lights stay within configured MIN_GREEN_TIME and MAX_GREEN_TIME bounds, and mandates a yellow transition phase before switching sides.
- **Early-Clearance (Adaptive) Rule:** If traffic on the current active side falls below a specific threshold during the wait period, the system applies queue-aware adaptive logic to switch the light early, preventing dead time.
- **Emergency Priority Rule:** Grants immediate priority to ambulances. The normal signal cycle is preempted to allow safe passage and safely resumes the interrupted cycle once clear.
- **Safe Movement Rule:** Vehicles near the stop-line must hold on RED, maintain minimum following distances, while through-vehicles are permitted to continue.
- **Violation Capture Rule:** Side-lane or middle-lane violations are logged strictly once per vehicle when relevant criteria are breached.

B) Implementation of Validation Logic

Validation logic is heavily implemented across the system to ensure stability, safety, and correct data formatting before processing:

- **Input/Resource Validation:** The system verifies video file existence and performs open-checks before initiating processing to prevent file-read crashes.
- **Detection Filtering:** Raw YOLO detection outputs are validated against a confidence threshold; only objects meeting this threshold are accepted as valid vehicles.
- **Runtime Data Safety:** The system validates incoming frames. If a frame is empty or None, it returns safe default detection dictionaries to prevent key errors during runtime.
- **Controller Argument Validation:** When traffic personnel use the manual_override (target_side) function, the system validates the target side against the allowed sequence to prevent illogical signal states.
- **Database Operation Safety:** Violation logging wraps database operations in try/except blocks. If the MySQL database fails or disconnects, the validation catches the error, preventing the main traffic application from crashing.

C) Data Transformation Strategies

Data retrieved from the data/model layer is continuously transformed into formats consumable by the Presentation Layer (UI):

- **AI Output to Traffic Metrics:** Raw YOLO model outputs (bounding boxes, class IDs, raw confidence scores) are transformed into clean, UI-friendly metrics (total counts, specific vehicle types, and emergency flags).
- **Frame Transformation:** Raw OpenCV BGR video frames are mathematically transformed into RGB format, and then converted into PIL ImageTk objects so they can be rendered live in the Tkinter UI.
- **Internal State to Visual State:** The BLL's internal enumerations and raw integer timers are mapped and transformed into user-friendly visual indicators, such as specific color hex codes (red/yellow/green) and formatted countdown strings.
- **Simulation Physics to Rendered Scene:** Raw backend data like vehicle coordinates, headings, and queue states are transformed into rotated visual sprites, UI labels, and statistics panel values in the Pygame window.
- **Event Data Formatting:** Raw runtime events are aggregated and transformed into formatted CSV/text logs for easy viewing by system administrators.